



Code Profiling

2025

André Pereira

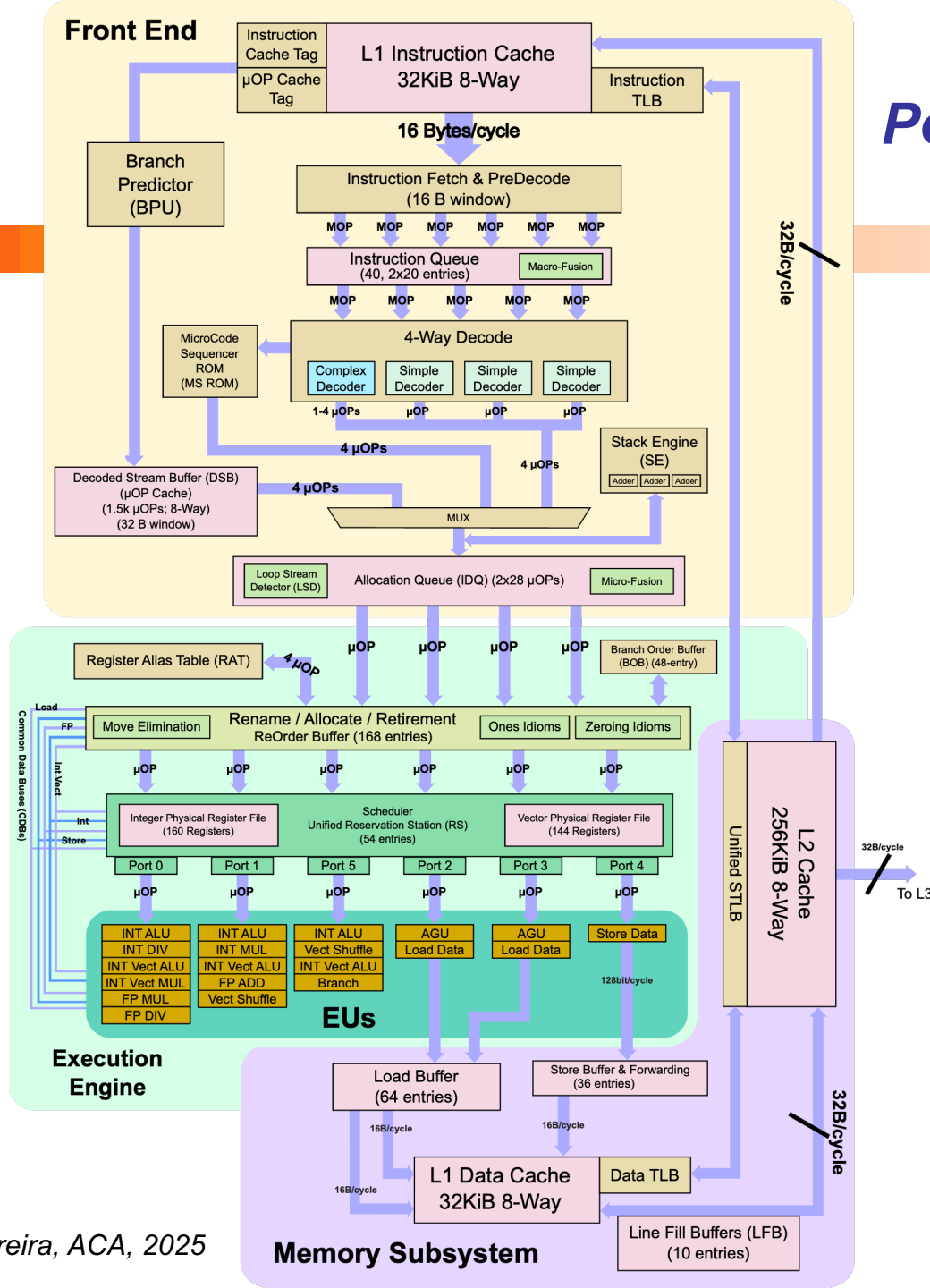


- **Why it matters?**
 - Understanding program behaviour in a high-performance context
 - Identifying performance bottlenecks and inefficiencies
- **Goals**
 - Optimise code runtime performance metrics (not only T_{exec} !)
 - Characterise multi-node, multi-core, and accelerator architectures
- **Context in HPC**
 - Role in achieving cost-effective computing utilisation
 - Impact on scientific simulations and large-scale data processing

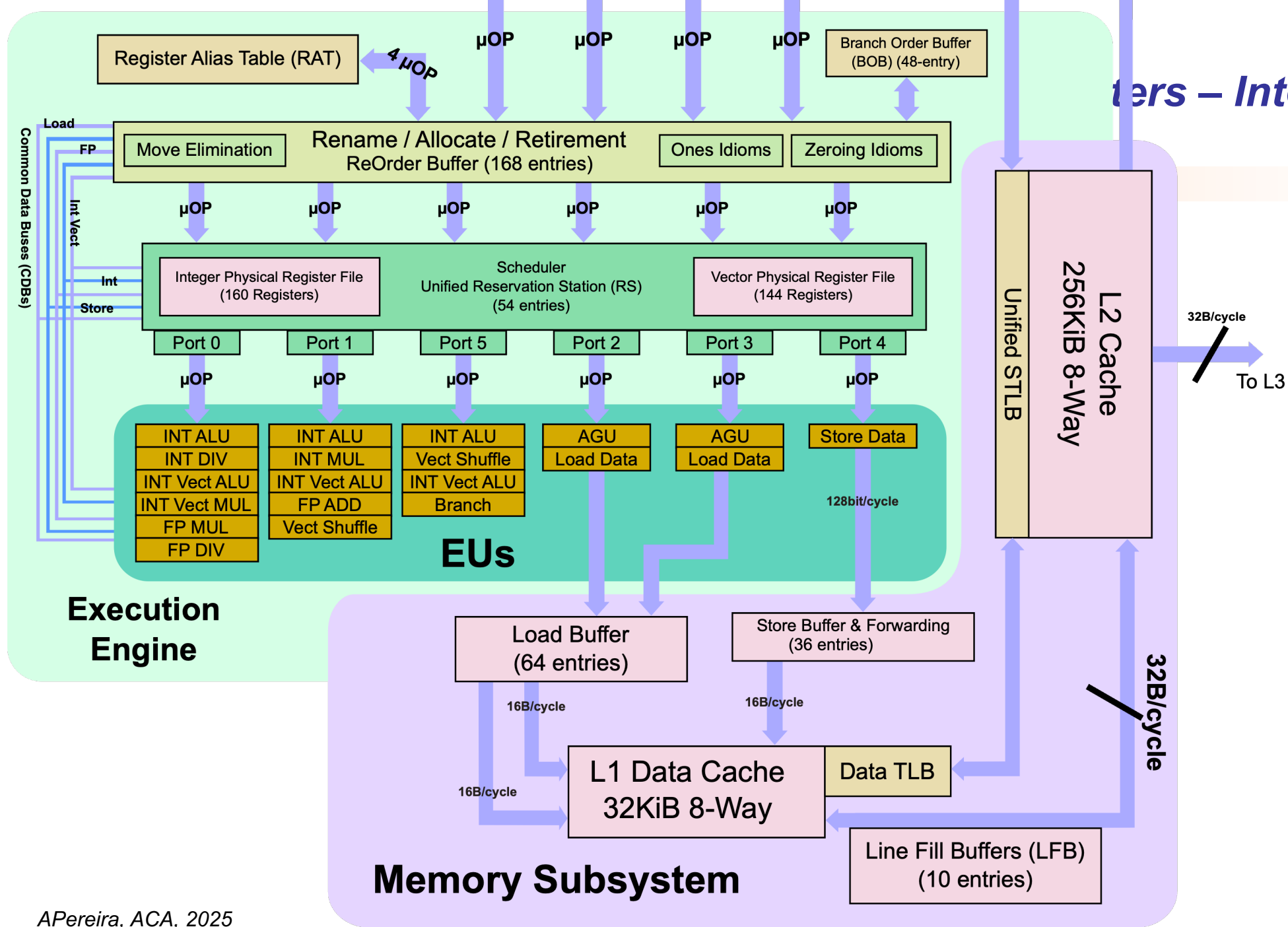


- **What for?**
 - Registers that track low-level events during program execution
 - Provide insights into device activity and system behaviour
- **Enable fine-grained analysis**
 - Identify performance bottlenecks at the microarchitectural level
 - But this level of detail is not required all the time
 - Direct tuning efforts for code and system configurations
- **How do we use them?**
 - PAPI facilitate access to hardware counters, integrating with other profilers

Performance Counters – Intel Ivy Bridge



Processors – Intel Ivy Bridge





- **Some considerations**
 - Minimal impact on performance
 - Excessive monitoring can perturb the system
 - A limited number of (concurrent) counters per device
 - Requires multiplexing for comprehensive data collection
 - Different architectures offer varying counters and capabilities
 - Affecting cross-platform comparisons

Instrumentation Methods – Static Instrumentation



- Code is modified during compilation
 - Embed measurement hooks directly in the application
 - Example: Compiler flags for inserting profiling routines
- Advantages
 - Low runtime overhead; highly precise since instrumentation is planned
 - Ensures consistent data collection in every measurement
- Disadvantages
 - Requires recompilation
 - May not capture runtime dynamic behaviours

Instrumentation Methods – Dynamic Instrumentation

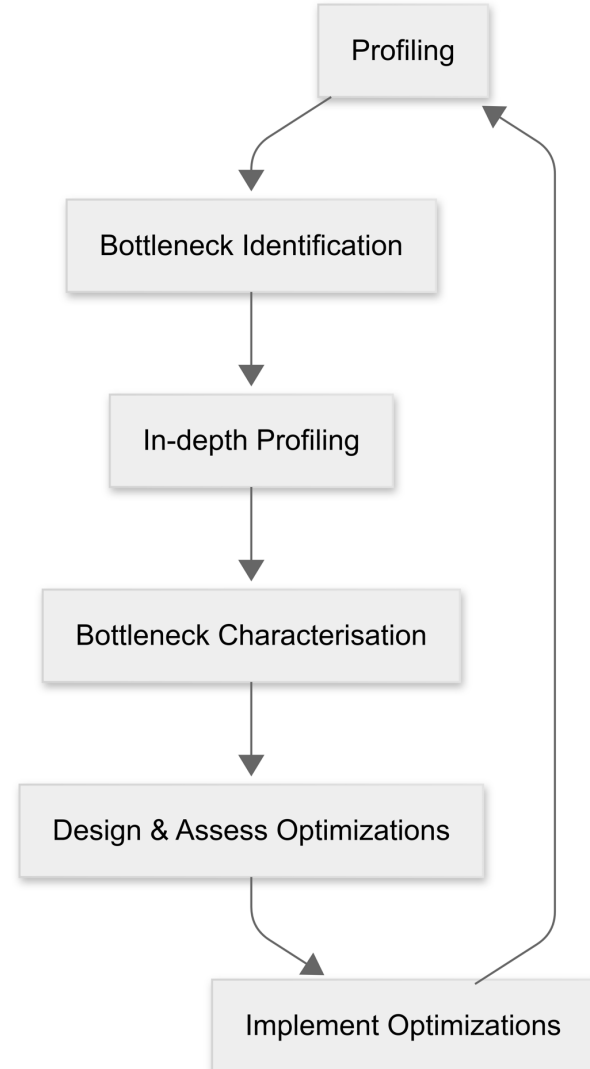


- Runtime Modification
 - Instrumentation is added or removed while the application is running
 - Tools (like VTune) can inject probes without recompiling
- Advantages
 - Flexibility to enable/disable instrumentation on the fly
 - Useful for monitoring long-running or production applications
- Disadvantages
 - Potentially higher overhead compared to static methods
 - Can be more complex to analyse behaviour correctly



- Sampling
 - Periodically captures the program's state (e.g., call stacks, performance counters)
 - **Pros:** Low overhead; suitable for large-scale applications
 - **Cons:** May miss short-lived events
- Tracing
 - Records every event, providing a detailed execution log
 - **Pros:** Comprehensive insight into application behaviour
 - **Cons:** Can generate massive data sets and incur high overhead

The Profiling Workflow





- **Sampling-based Profilers**
 - gprof, Intel VTune Amplifier
- **Event-based Profilers**
 - PAPI (Performance API), perf, TAU
- **MPI Profilers**
 - mpiP, Intel Trace Analyzer and Collector, score-p
- **GPU Profilers**
 - NVIDIA Nsight, AMD CodeXL



- Analyses CPU utilisation, threading efficiency, memory usage, and I/O behaviour
 - Identifies critical sections of code that consume the most resources
 - Evaluates performance on multi-core and multi-threaded systems
- Supports both sampling-based and event-based profiling methods
- Compatible with x86 devices (AMD and Intel)
 - But not with ARM...
- **A lot of training material online!**



- Important types of analyses (quality depends on uarch)
 - hotspots
 - threading (when complex thread management is implemented)
 - hpc-performance
 - memory-consumption
 - memory-access
 - uarch-exploration (mostly relates to hardware characterisation)
- **Experiment to get familiarised!**

VTune – CLI Sample Outputs



```
Effective Physical Core Utilization: 2.5% (0.493 out of 20)
| The metric value is low, which may signal a poor physical CPU cores
| utilization caused by:
|   - load imbalance
|   - threading runtime overhead
|   - contended synchronization
|   - thread/process underutilization
|   - incorrect affinity that utilizes logical cores instead of physical
|     cores
| Explore sub-metrics to estimate the efficiency of MPI and OpenMP parallelism
| or run the Locks and Waits analysis to identify parallel bottlenecks for
| other parallel runtimes.
|
Effective Logical Core Utilization: 2.5% (0.986 out of 40)
| The metric value is low, which may signal a poor logical CPU cores
| utilization. Consider improving physical core utilization as the first
| step and then look at opportunities to utilize logical cores, which in
| some cases can improve processor throughput and overall performance of
| multi-threaded applications.
```

VTune – CLI Sample Outputs



```
CPU Time: 17.240s
Memory Bound: 0.0% of Pipeline Slots
  L1 Bound: 19.2% of Clockticks
  L2 Bound: 0.0% of Clockticks
  L3 Bound: 14.3% of Clockticks
  DRAM Bound: 0.0% of Clockticks
    DRAM Bandwidth Bound: 0.0% of Elapsed Time
  Store Bound: 0.5% of Clockticks
  NUMA: % of Remote Accesses: 0.0%
Loads: 49,336,239,048
Stores: 16,951,636,212
LLC Miss Count: 0
  Local DRAM Access Count: 0
  Remote DRAM Access Count: 0
  Remote Cache Access Count: 0
Average Latency (cycles): 8
Total Thread Count: 2
Paused Time: 0s
```

Bandwidth Utilization

Bandwidth Domain	Platform Maximum	Observed Maximum	Average	% of Elapsed Time with High BW Utilization(%)
-----	-----	-----	-----	-----
DRAM, GB/sec	96	5.200	3.918	0.0%
DRAM Single-Package, GB/sec	48	5.100	2.977	0.0%

VTune – Graphical User Interface

 **HPC Performance Characterization** HPC Performance Characterization ? 

 Analysis Configuration Collection Log **Summary** Bottom-up

 **Elapsed Time[®]: 18.818s**

SP GFLOPS[®]: 0.000
DP GFLOPS[®]: 0.000
x87 GFLOPS[®]: 0.000
CPI Rate[®]: 0.711
Average CPU Frequency[®]: 2.9 GHz
Total Thread Count: 1

 **Effective Physical Core Utilization[®]: 2.3% (0.453 out of 20) ↗**

Effective Logical Core Utilization[®]: 2.3% (0.907 out of 40) ↗

 **Effective CPU Utilization Histogram**

 **Memory Bound[®]: 0.0% of Pipeline Slots**

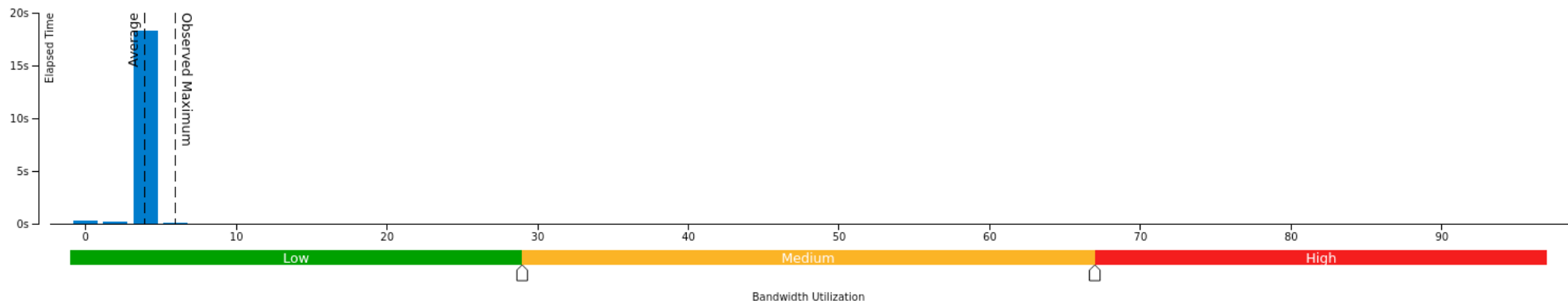
Cache Bound[®]: 0.0% of Clockticks
 **DRAM Bound[®]: 0.0% of Clockticks**

DRAM Bandwidth Bound[®]: 0.0% of Elapsed Time
NUMA: % of Remote Accesses[®]: 0.0%

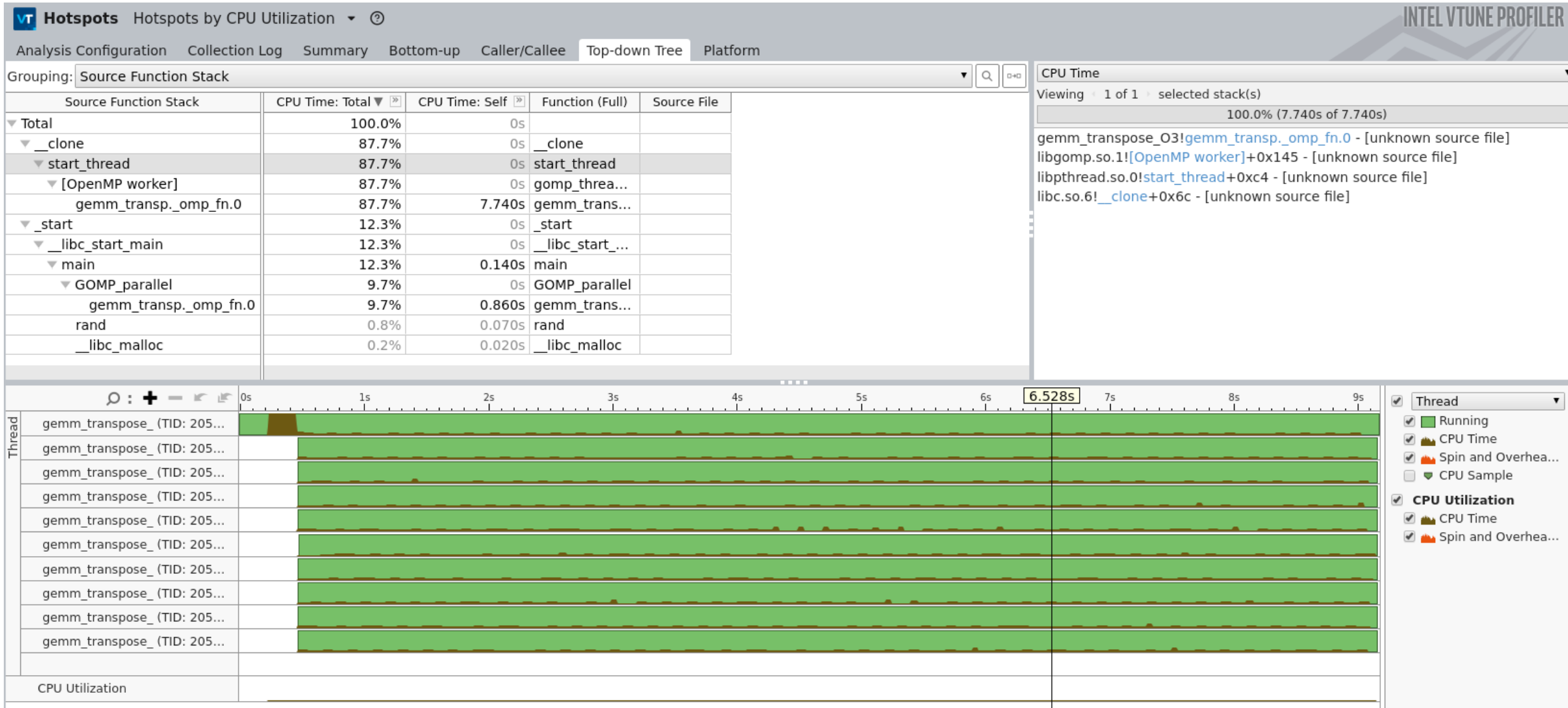
 **Bandwidth Utilization Histogram**

Explore bandwidth utilization over time using the histogram and identify memory objects or functions with maximum contribution to the high bandwidth utilization.
Bandwidth Domain:
 **Bandwidth Utilization Histogram**

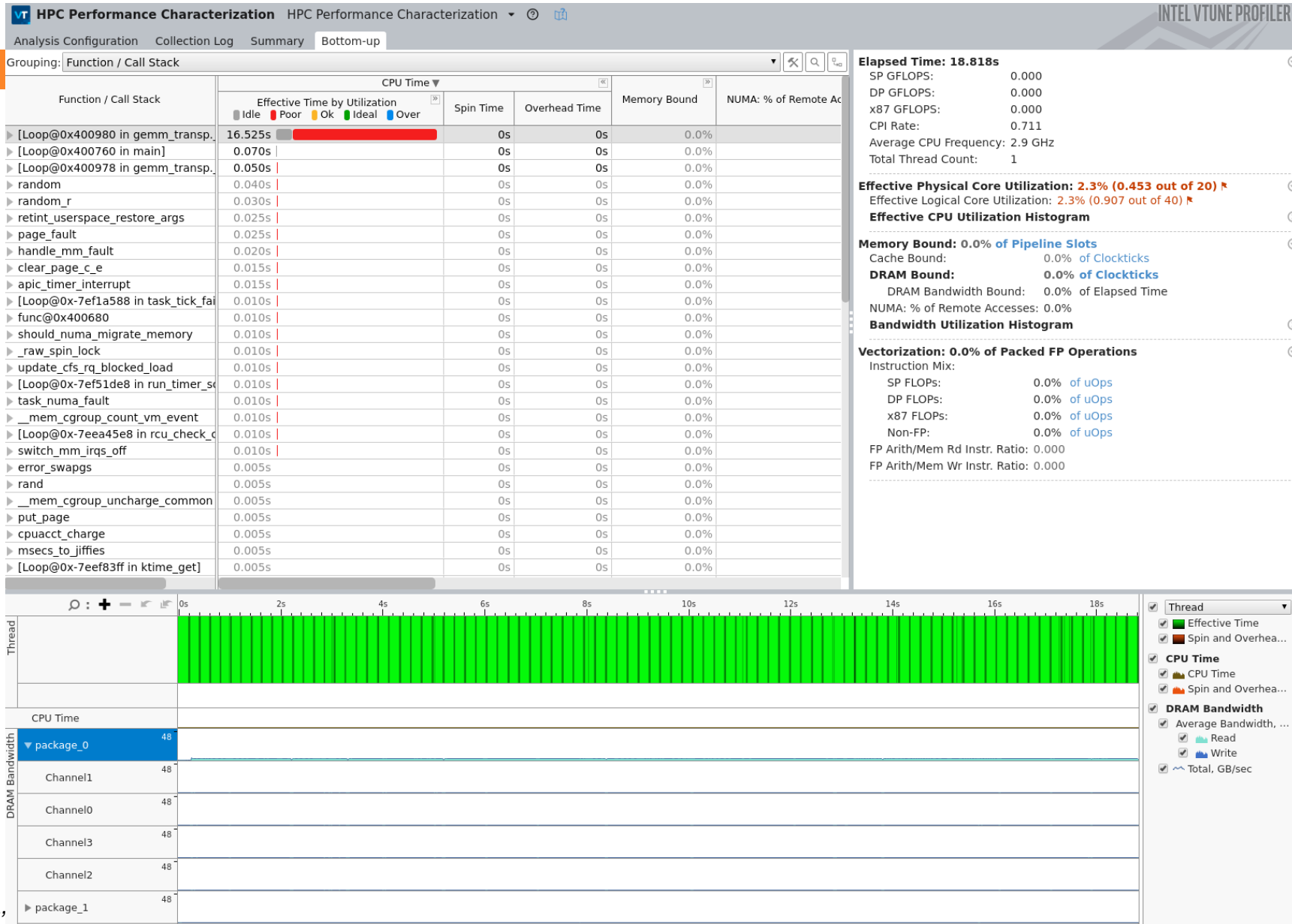
This histogram displays the wall time the bandwidth was utilized by certain value. Use sliders at the bottom of the histogram to define thresholds for Low, Medium and High utilization levels. You can use these bandwidth utilization types in the Bottom-up view to group data and see all functions executed during a particular utilization type. To learn bandwidth capabilities, refer to your system specifications or run appropriate benchmarks to measure them; for example, Intel Memory Latency Checker can provide maximum achievable DRAM and Interconnect bandwidth.



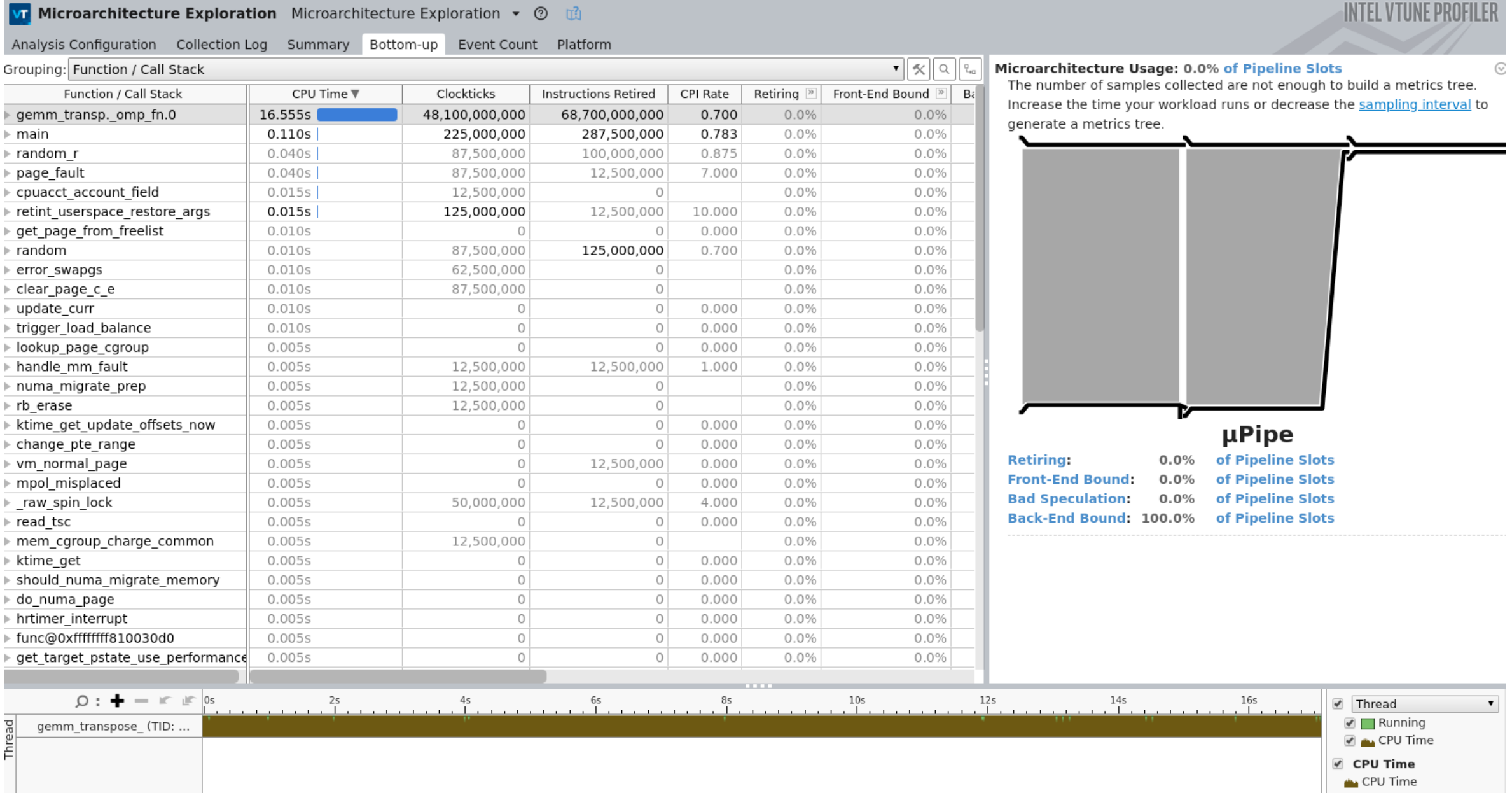
VTune – Graphical User Interface



VTune – Graphical User Interface



VTune – Graphical User Interface



VTune – Graphical User Interface

 **Memory Access** Memory Usage ▾ ⓘ 📄

Analysis Configuration Collection Log **Summary** Bottom-up Platform

INTEL VTUNE PROFILE



⌵ **Elapsed Time[®]: 19.448s**

CPU Time[®]: 17.240s

⌵ **Memory Bound[®]: 0.0%** of Pipeline Slots

Loads: 49,336,239,048

Stores: 16,951,636,212

⌵ **LLC Miss Count[®]: 0**

Average Latency (cycles)[®]: 8

Total Thread Count: 2

Paused Time[®]: 0s

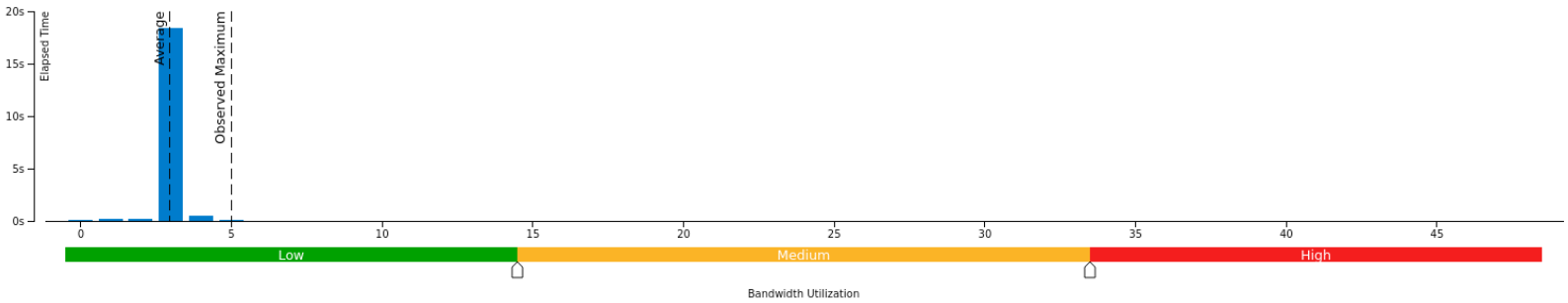
⌵ **Bandwidth Utilization Histogram**

Explore bandwidth utilization over time using the histogram and identify memory objects or functions with maximum contribution to the high bandwidth utilization.

Bandwidth Domain: **DRAM Single-Package, GB/sec ▾**

⌵ **Bandwidth Utilization Histogram**

This histogram displays the wall time the bandwidth was utilized by certain value. Use sliders at the bottom of the histogram to define thresholds for Low, Medium and High utilization levels. You can use these bandwidth utilization types in the Bottom-up view to group data and see all functions executed during a particular utilization type. To learn bandwidth capabilities, refer to your system specifications or run appropriate benchmarks to measure them; for example, Intel Memory Latency Checker can provide maximum achievable DRAM and Interconnect bandwidth.



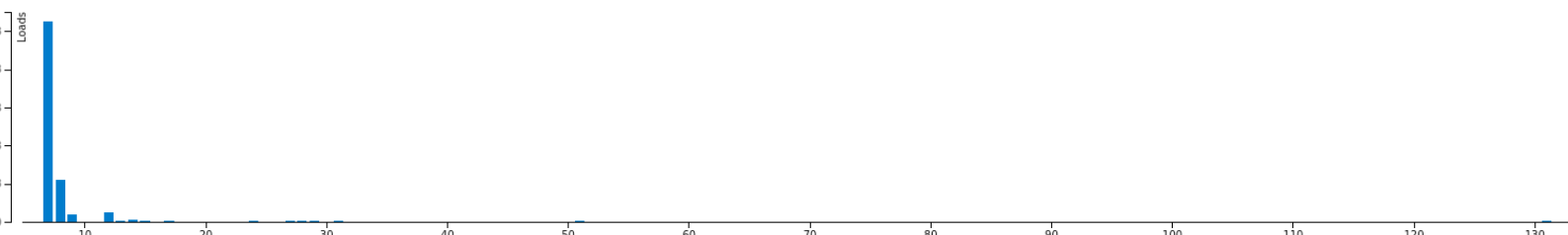
⌵ **Top Functions with High Bandwidth Utilization**

This section shows top functions, sorted by LLC Misses that were executing when bandwidth utilization was high for the domain selected in the histogram area.

No data to show. The collected data is not sufficient.

⌵ **Latency Histogram**

This histogram shows a distribution of loads per latency (in cycles).



VTune – Graphical User Interface

